

Autonomous Cognitive Engine for Deep Research and Long-Horizon Tasks

ANUSHA BENNY

January 20, 2026

Outline

- 1 Project Overview
- 2 Project Objective
- 3 SystemArchitecture
- 4 Execution Workflow
- 5 Key Features
- 6 Sub-Agent Delegation
- 7 Observability with LangSmith
- 8 Results and Evaluation
- 9 Conclusion and Future Scope

Project Overview

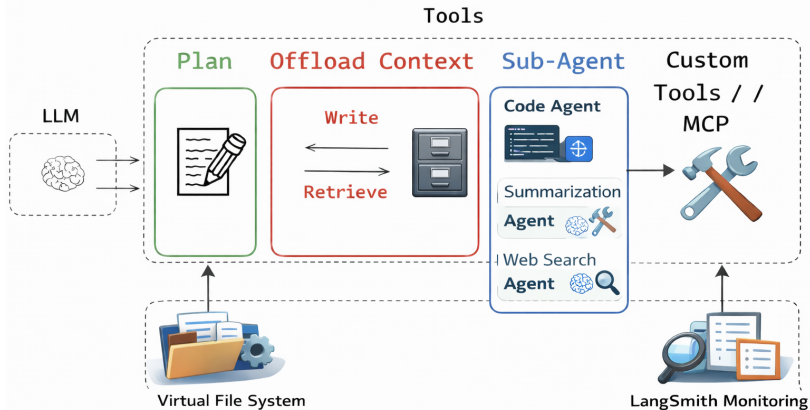
- Builds an autonomous AI system for complex, long-horizon task execution
- Uses a **Supervisor Agent** for planning and execution control
- Supports **task decomposition** through structured TODO planning
- Introduces **persistent memory** using a Virtual File System (VFS)
- Enables **multi-agent collaboration** for specialized tasks
- Provides **stateful execution and tracing** using LangGraph and LangSmith

Project Objective

- Build an **Autonomous Cognitive Engine**
- Support:
 - Structured task planning
 - Persistent memory
 - Multi-agent collaboration
 - Long-horizon execution
- Goal:

Enable autonomous execution of complex research and multi-step workflows using LLMs

SystemArchitecture



System Architecture

- Supervisor Agent (central controller)
- Planning Module (TODO generation)
- Virtual File System (persistent memory)
- Sub-Agents:
 - Code Agent
 - Summarization Agent
 - Web Search Agent
- LangGraph for stateful execution
- LangSmith for tracing and observability

Execution Lifecycle

Initialize → Plan → Reason → Act → Observe → Store → Synthesize

- User submits task
- Supervisor creates structured plan
- Executes steps iteratively
- Uses memory and sub-agents
- Produces final output

Key Features

- Structured Task Planning (TODO steps)
- Persistent Memory using Virtual File System
- Multi-Agent Delegation
- Calendar and Event Management
- Tool-based execution in LangSmith
- Full observability and tracing

Sub-Agent Delegation

- Supervisor detects task intent
- Delegates to specialized agents:
 - **Code Agent** – programming tasks
 - **Summarization Agent** – text condensation
 - **Web Search Agent** – external knowledge

Benefits

Modularity, higher accuracy, scalable architecture

Observability with LangSmith

- Every step is traced and monitored
- Visible components:
 - Planning steps
 - Tool executions
 - Sub-agent calls
 - LLM reasoning
- Enables:
 - Debugging
 - Evaluation
 - Reproducibility

Observability with LangSmith-Result

TRACE

Waterfall

ChatSession 2.12s

Delegation Decision 0.00s

Plan Requirement Check 0.00s

Auto Task Planning 0.94s

Groq-LLM-Call 0.93s

Task-Delegation-Tool 1.16s

Agent Lookup 0.00s

Sub-Agent Execution 1.16s

summarization 1.16s

Groq-LLM-Call 1.15s

ChatSession

Run Feedback Metadata

Input

User Text

Artificial Intelligence (AI) agents are intelligent systems that are capable of observing their environment, making decisions, and performing actions to achieve specific objectives. Unlike traditional AI applications that only respond to user inputs, AI agents are designed to operate autonomously by continuously interacting with their surroundings and adapting their behavior based on feedback. An AI agent typically consists of a perception mechanism to receive input, a reasoning or decision-making component to analyze information, and an action module that allows it to execute tasks. With the advancement of large language models and intelligent frameworks, AI agents can now handle complex workflows such as planning, problem-solving, task execution, and collaboration with other agents. They can use memory to retain context, tools to access external systems, and logic to determine the most appropriate next action. AI agents are widely used in applications such as virtual assistants, recommendation systems, autonomous systems, educational platforms, and enterprise automation. By enabling systems to act intelligently and independently, AI agents play a crucial role in building scalable, efficient, and adaptive AI-driven solutions across various domains.

State

Conversation Id b6838695-dbb0-475b-8255-3f41bf72563f

Chat History [...]

Playground Thread Add to

START TIME

01/19/2026, 06:44:57 PM GMT+5:

END TIME

01/19/2026, 06:44:59 PM GMT+5:

STATUS

Success

TOTAL TOKENS

0 tokens

LATENCY

2.12s

TYPE

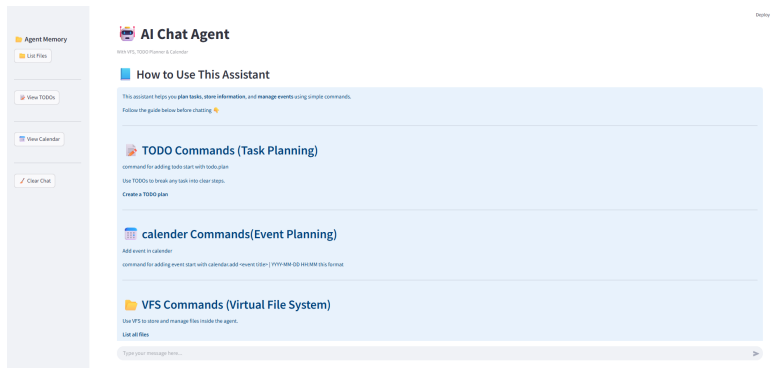
Chain

TAGS

chat main-session

Policy

User Interface



The system provides an interactive chat-based interface with a text input box, real-time agent responses, TODO planning, calendar management, and virtual file storage. Users can submit complex multi-step tasks and visually observe how the supervisor agent plans, delegates to sub-agents, and synthesizes results using persistent memory.

User Interface(TodoPlan)



todo.plan a event of birthdayparty



TODO created:

1. Send out invitations to guests 2 weeks prior to the event
2. Plan the party theme and decorations
3. Book a venue or decide on home setup
4. Create a guest list and send out RSVPs
5. Plan the party menu and order necessary supplies
6. Arrange for entertainment (music, games, etc.)
7. Prepare a photo booth or other interactive activities
8. Send reminders to guests 1 week prior to the event
9. Confirm the party schedule and timings with vendors
10. Prepare a gift table and plan for gift-giving



calendar.add birthdayparty|2026-09-21 12:00



Event added: birthdayparty → 2026-09-21 12:00



write birthday invitation invite friends and family



Wrote to birthday

User Interface(summarize Agent)



summarize dsa topics



Delegated to **summarize agent**:

DSA (Data Structures and Algorithms) topics typically include:

1. **Arrays**: Basic operations, searching, sorting, and manipulation.
2. **Linked Lists**: Singly and doubly linked lists, insertion, deletion, and traversal.
3. **Stacks**: Last-In-First-Out (LIFO) data structure, operations like push, pop, and peek.
4. **Queues**: First-In-First-Out (FIFO) data structure, operations like enqueue and dequeue.
5. **Trees**: Binary trees, tree traversal (inorder, preorder, postorder), and operations like insertion and deletion.
6. **Graphs**: Representation, traversal (BFS, DFS), and operations like shortest path and minimum spanning tree.
7. **Sorting Algorithms**: Bubble sort, selection sort, insertion sort, merge sort, and quick sort.
8. **Searching Algorithms**: Linear search, binary search, and hash table search.
9. **Hashing**: Hash table implementation, collision resolution, and operations like insert and search.
10. **Greedy Algorithms**: Solving problems using the greedy approach, such as Huffman coding and activity selection.
11. **Dynamic Programming**: Solving problems using memoization and tabulation, such as Fibonacci series and longest common subsequence.
12. **Bit Manipulation**: Operations like bitwise AND, OR, and XOR, and their applications.

User Interface(code Agent)



give me the code of sum of two numbers in java



Delegated to code agent:

Sum of Two Numbers in Java

Here is a simple Java program that calculates the sum of two numbers:

```
public class SumOfTwoNumbers {  
    /**  
     * Method to calculate the sum of two numbers.  
     */  
    *  
    * @param num1 The first number.  
    * @param num2 The second number.  
    * @return The sum of num1 and num2.  
    */  
    public static int calculateSum(int num1, int num2) {  
        // Check if both numbers are integers  
        if (num1 instanceof Integer && num2 instanceof Integer) {  
            // Calculate the sum  
            int sum = num1 + num2;  
            return sum;  
        } else {  
            // If either number is not an integer, throw an exception  
            throw new IllegalArgumentException("Both numbers must be integers.");  
        }  
    }  
}  
  
public static void main(String[] args) {  
    // Test the method with two integers  
    int num1 = 5;  
    int num2 = 10;  
    int sum = calculateSum(num1, num2);  
    System.out.println("The sum of " + num1 + " and " + num2 + " is: " + sum);  
}
```

User Interface(WebSearch Agent)



search the array implementation



Delegated to web_search agent:

1. **Searching Elements in an Array | Array Operations** In an unsorted array, the search operation can be performed by linear traversal from the first element to the last element, ie Linear Search. <https://www.geeksforgeeks.org/dsa/searching-elements-in-an-array-array-operations/>
2. **Binary Search in Array - algorithm** Binary search is an optimized solution for searching an element in an array as it reduces search time by following three ways. <https://stackoverflow.com/questions/249392/binary-search-in-array>
3. **Implementing binary search of an array (article)** Let's see how to think about binary search on a sorted array. Yes, JavaScript already provides methods for determining whether a given element is in an array. <https://www.khanacademy.org/computing/computer-science/algorithms/binary-search/a/implementing-binary-search-of-an-array>
4. **7.1.1. Searching in an Array** Sequential search is the best that we can do when trying to find a value in an unsorted array. [1] But if the array is sorted in increasing order by value, then ... <https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/BinarySearch.html>

Results and Evaluation

- Successfully handled:
 - Multi-step research tasks
 - Sub-agent delegation
 - Calendar scheduling
 - Persistent memory storage
 - LangSmith tracing for execution monitoring and debugging
- All test cases passed using Pytest
- Stable execution over long workflows
- End-to-end observability achieved through LangSmith logs

Conclusion

- Demonstrated a working autonomous multi-agent system
- Supports planning, memory, delegation, and synthesis
- Suitable for research and production systems

Future Scope

- More specialized sub-agents
- Semantic memory retrieval
- Human-in-the-loop supervision
- Distributed and scalable execution

Thank You

Questions?